

CivicPulse AI

Repository Structure & Feature-Wise Team Development Workflow

Project Context: Foundation 1 Status

We have successfully completed Foundation 1 (Day 1). This means our **project infrastructure** is in place: React + Tailwind, Node.js + Express, the Python AI placeholder, PostgreSQL + PostGIS, Docker Compose, shared docs, and our GitHub workflow.

Important: Issue Reporting business logic, Issue database tables, APIs, media upload, and map functionalities are NOT yet implemented. The structure exists, but the features do not.

1. Feature-Wise Team Ownership

To ensure clear ownership and avoid overlapping code conflicts, the project is divided into **5 DOMAIN FEATURES/MODULES**. Each member owns their complete feature top-to-bottom.

DON'T view roles by layer:

- Frontend Member
- Backend Member
- Database Member
- AI Member
- DevOps Member

DO view roles by domain:

- **Member 1 (Urvesh):** Issue Reporting
- **Member 2:** AI Issue Analysis
- **Member 3:** Duplicate + Verification
- **Member 4:** Priority + Authority
- **Member 5:** Dashboard + Analytics Map

Because each member owns a feature, you will work across the React frontend, Node.js backend, PostgreSQL database, and API integration as required by your specific domain.

2. Shared Repository Model

All 5 members will work inside **ONE common repository**. We will *not* create separate projects or repositories for each feature.

```
civicpulse-ai/
├─ client/           # React + Tailwind frontend
├─ server/           # Node.js + Express backend
├─ ai/               # Python AI service
├─ docs/             # Shared project documentation
├─ Doc/              # Reference PDFs
├─ docker-compose.yml # Local development stack
├─ .env.example       # Environment variables template
├─ .gitignore         # Ignored files
└─ README.md          # Project overview
```

3. How Features Map to the Repository

While the technical layers (client, server, ai) are shared, each feature gets its own domain/module area *inside* these layers. As you begin development, we recommend organizing your code conceptually like this:

```
client/
├─ src/
│   └─ features/          <-- Recommended Internal Organization
│       ├── issue-reporting/
│       ├── ai-analysis/
│       ├── duplicate-verification/
│       ├── priority-authority/
│       └─ dashboard-analytics/
server/
├─ src/
│   └─ modules/           <-- Recommended Internal Organization
│       ├── issue-reporting/
│       ├── ai-analysis/
│       ├── duplicate-verification/
│       ├── priority-authority/
│       └─ dashboard-analytics/
```

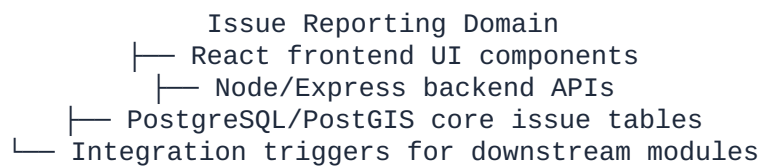
Note on Foundation 1

These feature folders do **not** currently exist in Foundation 1. You will create your respective domain folders as you begin implementing your features, while adhering to the shared architecture.

4. Feature Responsibilities

A. Issue Reporting — Urvesh (Member 1)

Responsibilities: Citizen reporting UI, step-by-step reporting form (Description, Category), Photo/media upload, GPS + map location, Review screen, Submission, Issue creation API, Issue database structure, Issue ID generation, Initial SUBMITTED status, Issue retrieval/tracking, and downstream Integration contracts.



B. AI Issue Analysis (Member 2)

Responsibilities: Image analysis, Category classification, Confidence scoring, and NLP/description processing.

C. Duplicate Detection + Community Verification (Member 3)

Responsibilities: Duplicate detection algorithms, Spatial similarity/proximity logic, and Community verification/upvoting functionality (where applicable).

D. Priority + Authority Management (Member 4)

Responsibilities: Priority calculation logic and Authority assignment/routing mechanisms.

E. Civic Dashboard + Analytics Map (Member 5)

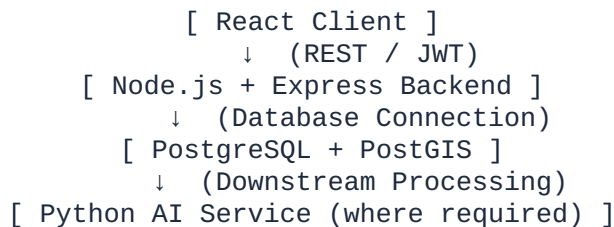
Responsibilities: Overall civic dashboard UI, statistical analytics, data visualization, and map-based presentation of issues.

5. Shared Architecture

Everyone builds upon the same foundational architecture. The rules are strict to ensure system stability:

- **Node.js is the primary API/orchestration layer.**

- **No Direct AI Calls:** The frontend does *not* directly call the Python AI service. It calls Node.js, which acts as the orchestrator.
- **Single Datastore:** PostgreSQL + PostGIS is the primary and only datastore.
- **Domain Boundaries:** Modules are logical boundaries inside the same application ecosystem.



6. How Modules Communicate

Issue Reporting is the entry point. It creates the central **Issue** entity. This entity is connected across all modules using a standardized **issue_id**.



Note: Processing does not have to be strictly synchronous. Modules execute downstream or asynchronously based on the architecture specification.

7. Shared Database Model

There is ONE shared PostgreSQL + PostGIS database. There is **not** one database per member.

- Each module owns its own domain data responsibilities.
- Members must **never** directly modify another module's core tables without team consensus.
- Shared database schemas and contracts must be closely coordinated, revolving around the central Issue entity.

```
users
  ↓
issues (Central Entity)
  ├── issue_media
  ├── issue_status_history
  └── [Other module-specific tables as designed]
```

8. Shared Documentation

The `docs/` folder contains the source of truth for our technical contracts:

- `API_CONTRACT.md`
- `ARCHITECTURE.md`
- `DATABASE_SCHEMA.md`
- `DATA_DICTIONARY.md`
- `ENUMS.md`
- `INTEGRATION_RULES.md`

Contract Enforcement

All members **MUST** follow these shared documents for naming conventions, enums, Issue ID formats, authentication, location formatting, and integration rules.

If a change is required: 1) Discuss with the team. 2) Agree on the change. 3) Update the shared documentation. 4) Only then, implement the change in code. No silent contract changes!

9. Git & GitHub Workflow

We use a standard branching strategy to protect the codebase:

- `main` → Stable/final release branch
- `develop` → Team integration branch
- `feature/*` → Individual member's working branch

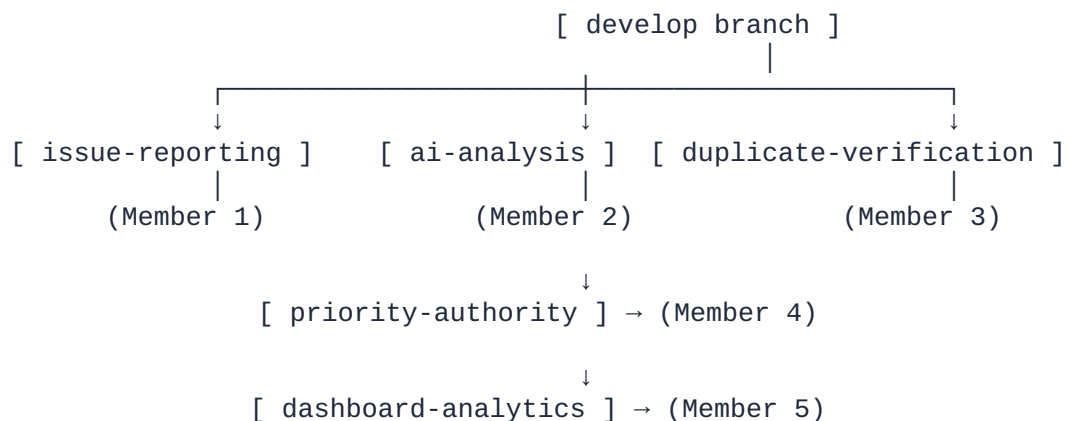
Step-by-Step Task Workflow:

1. `git checkout develop` & `git pull origin develop`

2. Create branch: `git checkout -b feature/<feature-name>-<task>`
(e.g., `feature/issue-reporting-database`, `feature/ai-analysis-api`)
3. Develop and test **only** on your feature branch.
4. Commit meaningful, descriptive changes.
5. Push feature branch to origin.
6. Create a Pull Request (PR): **Base = develop** | **Compare = feature/<task>**.
(Never point a feature PR directly to main!)
7. Team review.
8. Squash & Merge into `develop`.
9. Everyone pulls latest `develop` before starting new work.

10. Parallel Development Example

This structure allows all 5 members to work simultaneously without blocking each other or needing separate repositories:



11. Shared vs. Owned Responsibilities

SHARED (Team Responsibility)	FEATURE OWNED (Individual Responsibility)
• Repository Base • React & Node.js Setup • PostgreSQL/PostGIS & Docker Setup • JWT / Authentication standard • API Conventions & Enums • Core Issue Entity definition • Git Workflow & Environment variables	• Issue Reporting: Member 1 (Urvesh) • AI Analysis: Member 2 • Duplicate + Verification: Member 3 • Priority + Authority: Member 4 • Dashboard + Analytics: Member 5

12. Rules When Using AI Coding Tools

Since the team uses Antigravity / VS Code AI assistance, adhere strictly to these rules:

- AI **must** work within the existing repository structure.
- AI **must not** redesign the architecture without team approval.
- AI **must not** silently modify shared API contracts, duplicate entities, or alter shared enums.
- Generated code must be thoroughly reviewed, understood, and tested before committing.
- **Never commit secrets or .env files generated by AI.**
- Always prompt the AI from the perspective of your FEATURE, not just a technical layer. (e.g., *"Implement the Issue Reporting feature across the React frontend and Node.js backend according to the shared project contracts."*)

13. Continuous Integration Rule

We are **NOT** developing in isolation for several weeks and attempting a massive integration at the end. Integration is continuous.

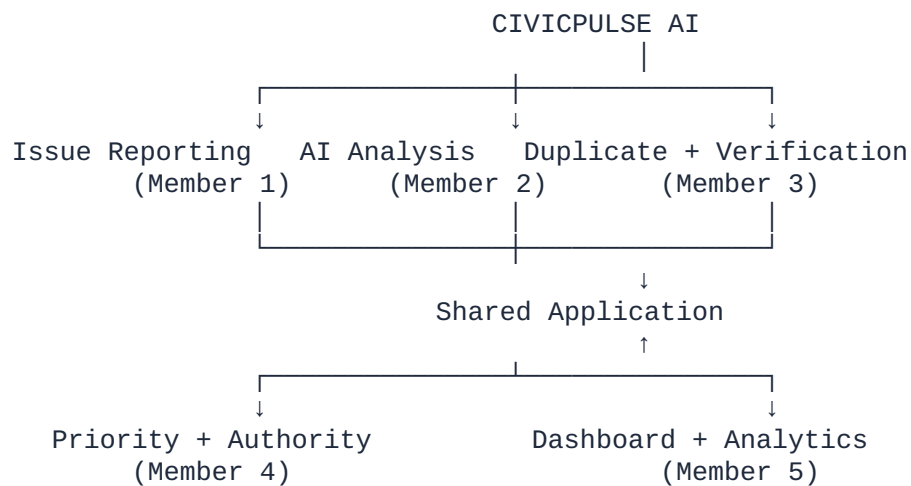
- Follow shared contracts rigorously.
- Pull the latest `develop` branch regularly.
- Test your features against the common environment.

- Raise contract conflicts early in team discussions.

14. The Beginner Mental Model

Keep this simple concept in mind at all times:

"One repository + one shared architecture + five feature owners."



All features use the same React + Node + PostgreSQL/PostGIS + Docker + GitHub environment.

15. Final Important Clarification

Foundation 1 created the common infrastructure. It did NOT create five independent, fully populated feature folders or complete modules.

The exact internal feature folder structure (like `client/src/features/issue-reporting/`) should be introduced gradually by each member as they begin implementation, taking care to preserve the overarching shared architecture.